

Computer Architecture

Course No. 21472

Week #6

Processor: Datapath and Control

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430

Introduction

CPU performance factors

- Instruction Count
- CPI and Cycle Time

In this Chapter, we will examine two MIPS implementation

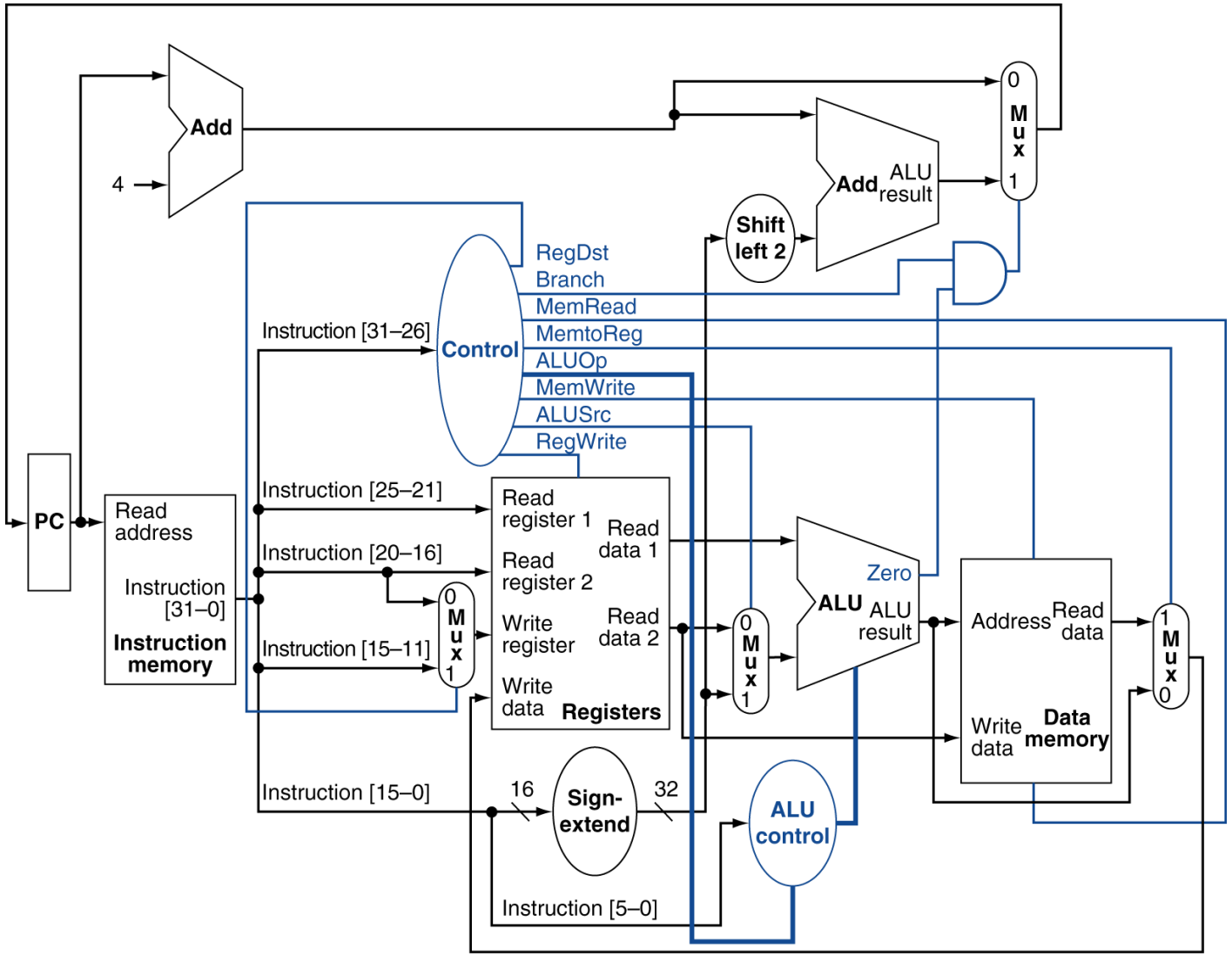
- A simplified version
- A more realistic pipelined version

Simple subset of Instructions

- Memory reference : `lw, sw`
- Arithemetic / Logical : `add, sub, and, or`
- Control transfer : `beq, j`

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

A Simplified Version of CPU



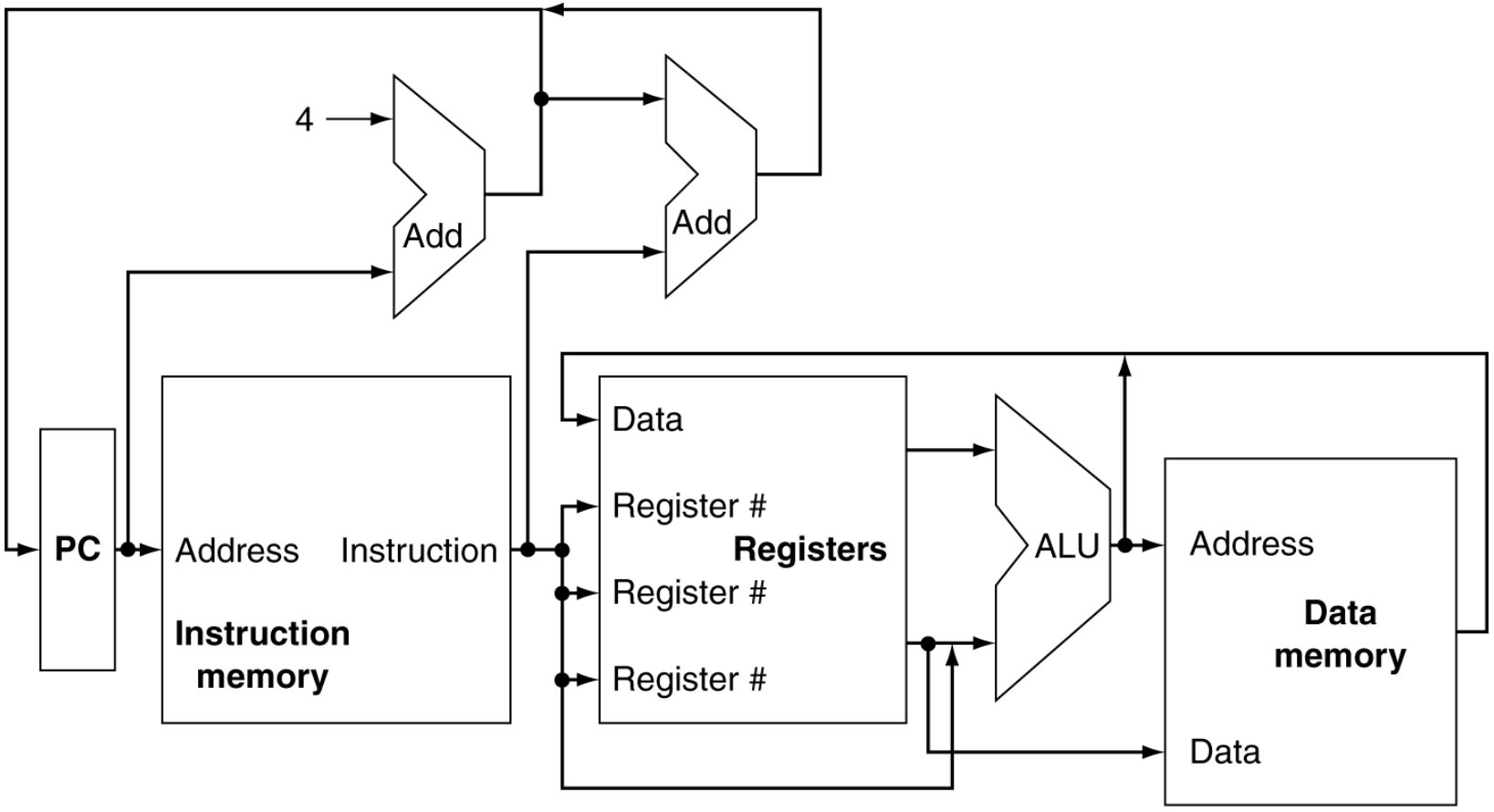
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Instruction Execution

- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← { target address or 'PC + 4' }
- Note that we are dealing with { `lw, sw, add, sub, and, or, slt, beq, j` }.

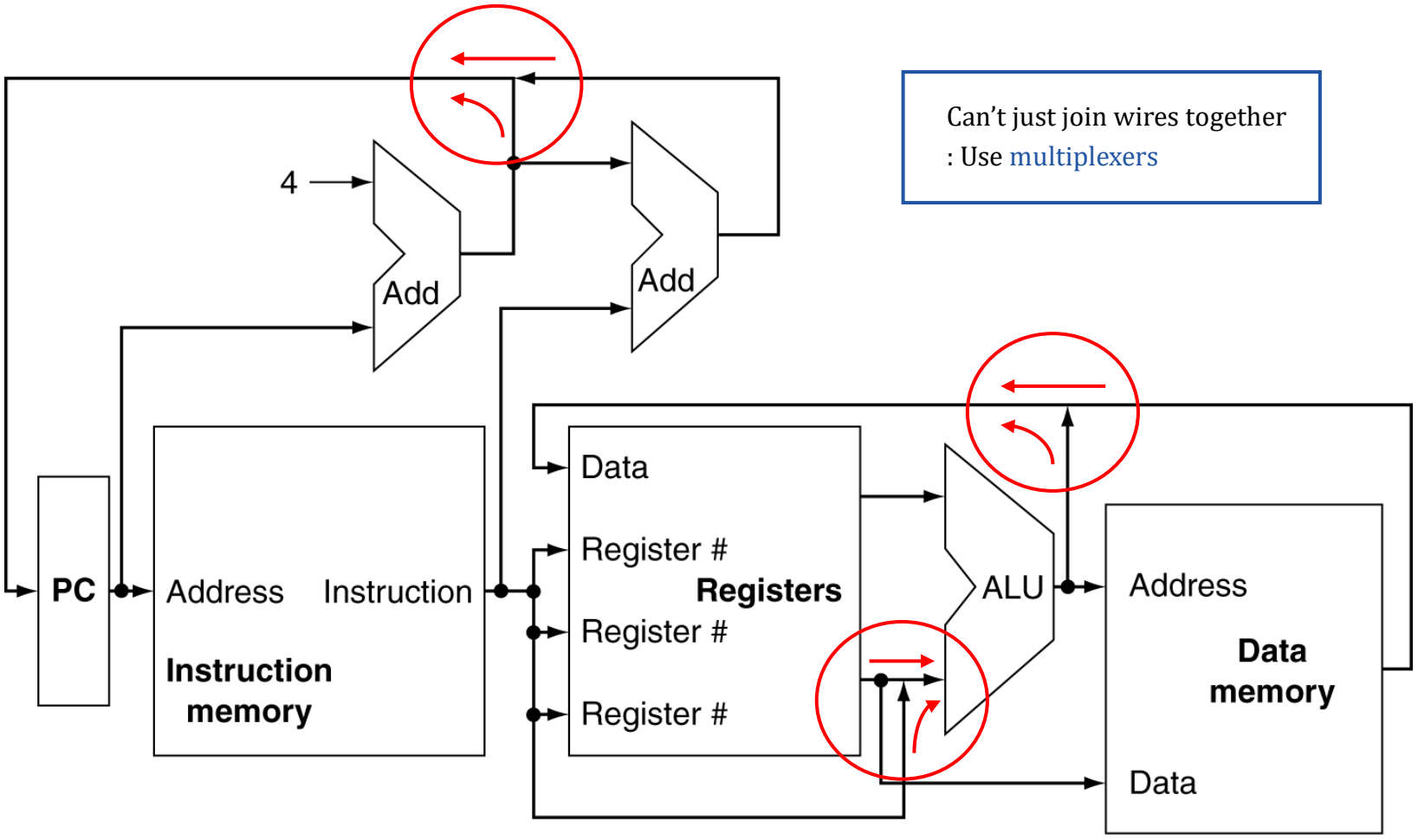
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPU Overview



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

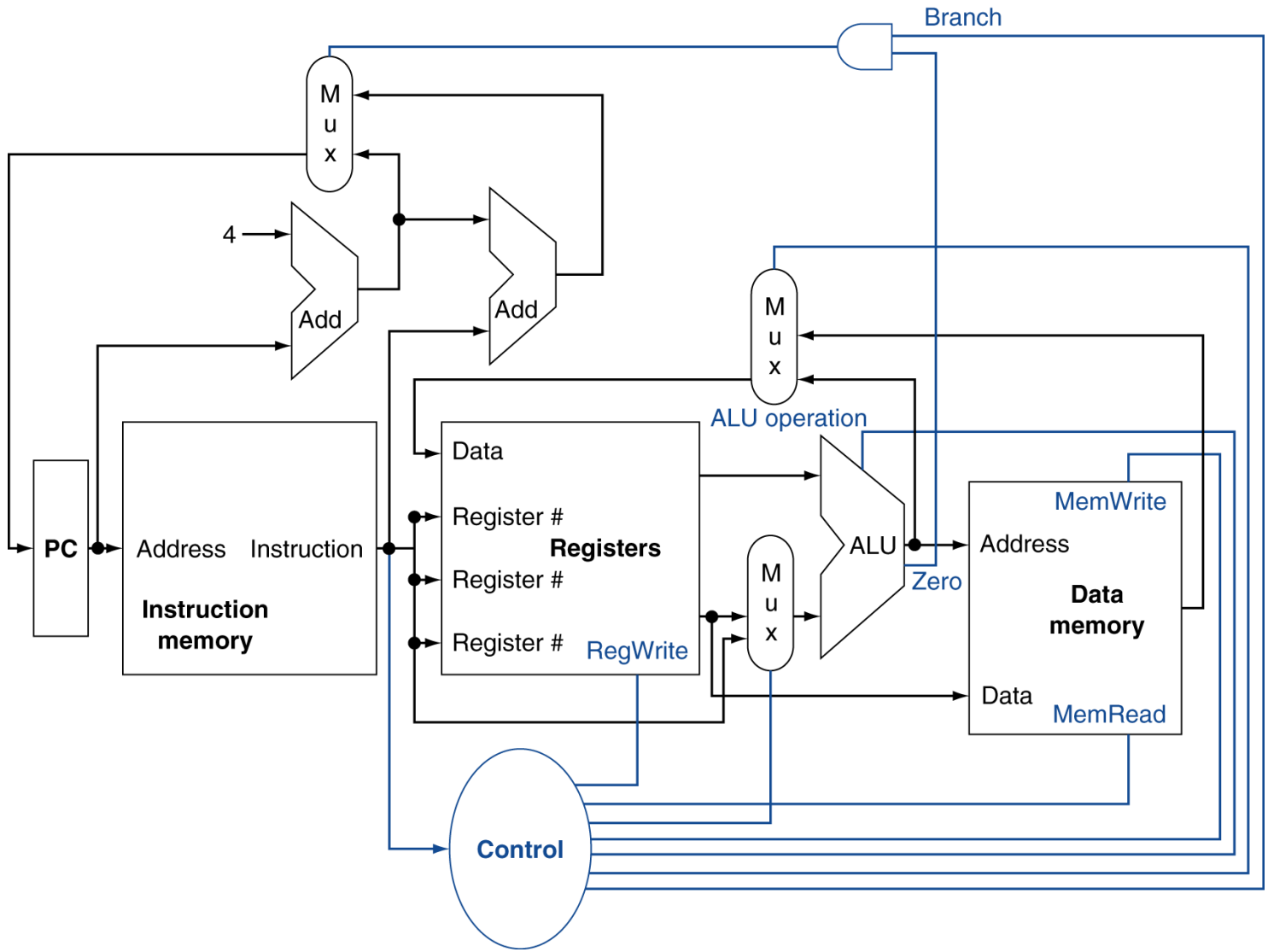
CPU Overview



Can't just join wires together
: Use multiplexers

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPU Overview: Control



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logic Design Basics

Information encoded in binary

- Low voltage = 0, High voltage = 1
- One wire per bit
- Multi-bit data encoded on multi-wire buses

Combinational element

- Operate on data
- Output is a function of input

State (sequential) Elements

- Store Information

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Combinational element

- Operate on data
- Output is a function of input

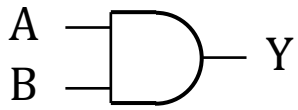
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logic Design Basics

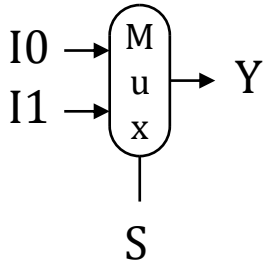
Combinational element

- Operate on data
- Output is a function of input

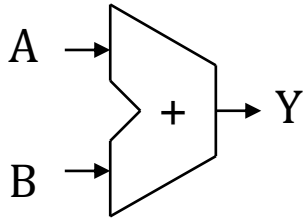
- AND-gate
  - Y = A & B



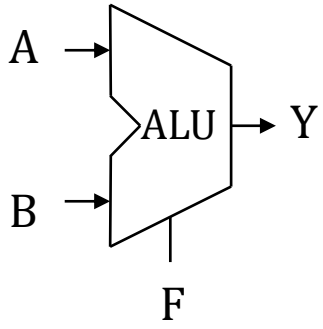
- Multiplexer
  - Y = S ? I1 : I0



- Adder
  - Y = A + B



- Arithmetic/Logic Unit
  - Y = F(A, B)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logic Design Basics

Information encoded in binary

- Low voltage = 0, High voltage = 1
- One wire per bit
- Multi-bit data encoded on multi-wire buses

Combinational element

- Operate on data
- Output is a function of input

State (sequential) Elements

- Store Information

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

State (sequential) Elements

- Store Information

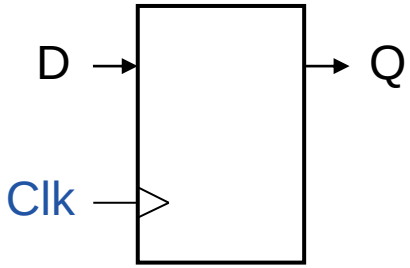
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

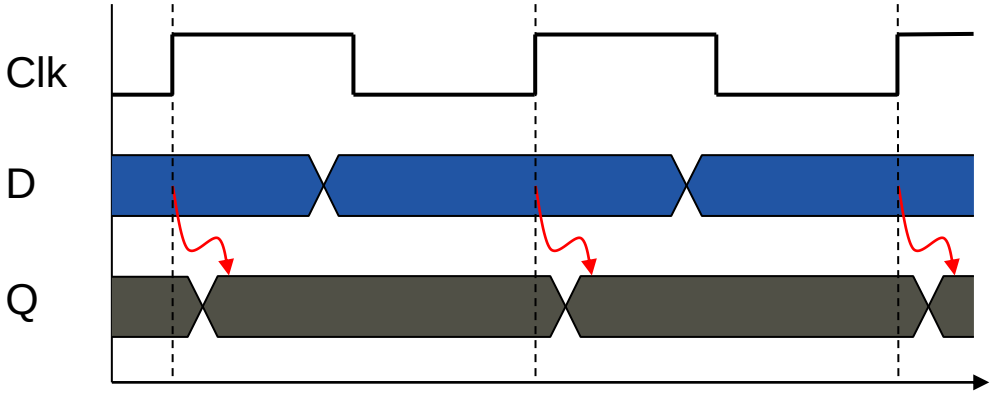
Sequential Elements

State (sequential) Elements

- Store Information

- **Register**: stores data in a circuit
  - Uses a clock signal to determine when to update the stored value
  - Edge-triggered: update when Clk changes from 0 to 1





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

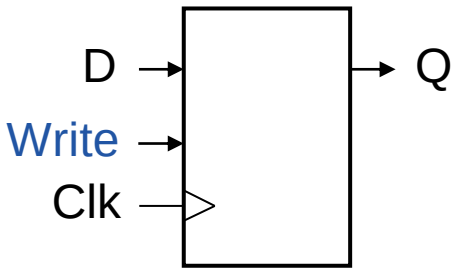
Sequential Elements

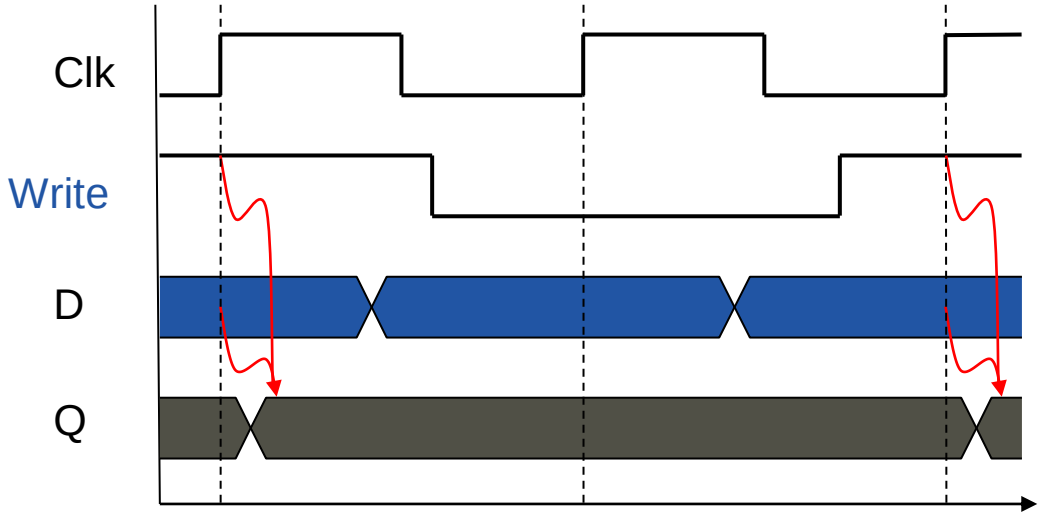
State (sequential) Elements

- Store Information

Register with write control

- Only updates on clock edge when write control input is 1
- Used when stored value is required later

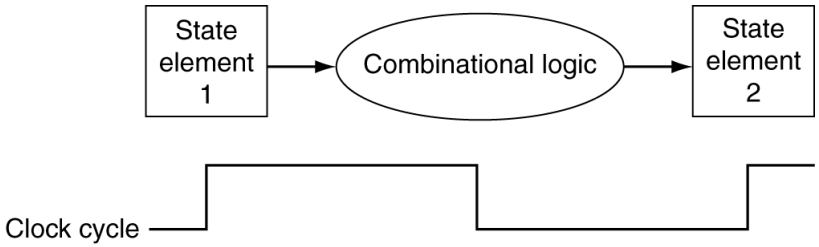


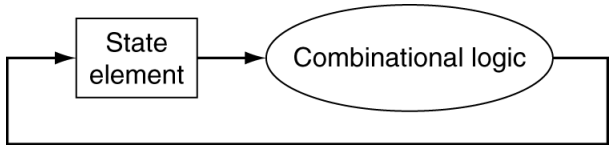


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Clocking Methodology

- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - Longest delay determines clock period





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Building a Datapath

- Datapath
  - Elements that process data and addresses in the CPU
    - Registers, ALUs, mux's, memories, …
- We will build a MIPS datapath incrementally
  - Just for a simple subset of the instructions
    - `lw, sw`
    - `add, sub, and, or, slt`
    - `beq, j`
  - Refining the overview design

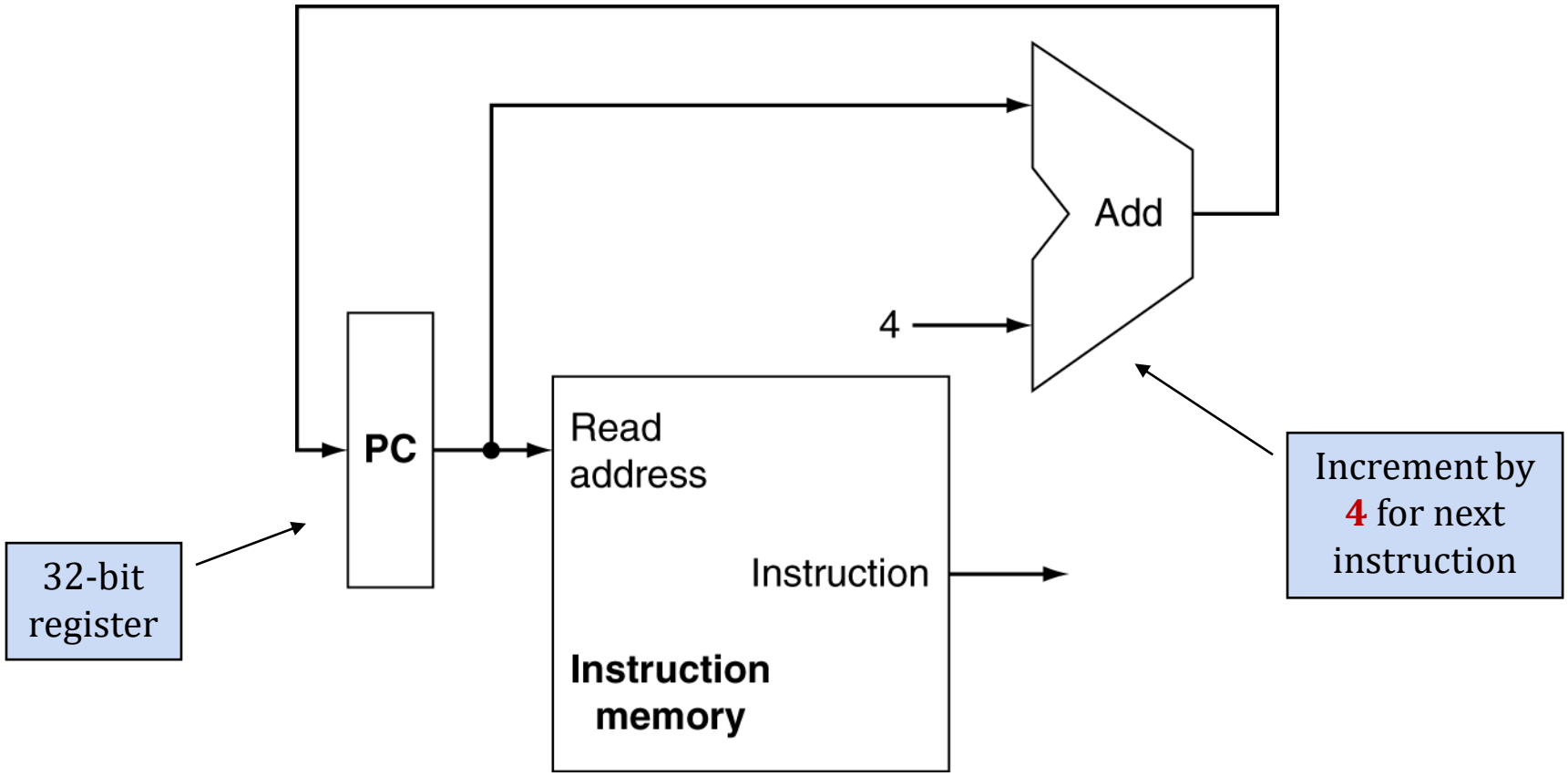
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retro: Instruction Execution

- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← { target address or 'PC + 4' }
- Note that we are dealing with { `lw, sw, add, sub, and, or, slt, beq, j` }.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Instruction Fetch



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: R-Format Instructions

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- Instruction fields
  - **op** : operation code (opcode) – always 0 in R-format because the operation is determined by 'funct'.
  - **rs** : first source register number
  - **rt** : second source register number
  - **rd** : destination register number
  - **shamt** : shift amount (00000 for now)
  - **funct** : function code (extends opcode)

- Example

```
add $t0, $s1, s2
```

000000	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

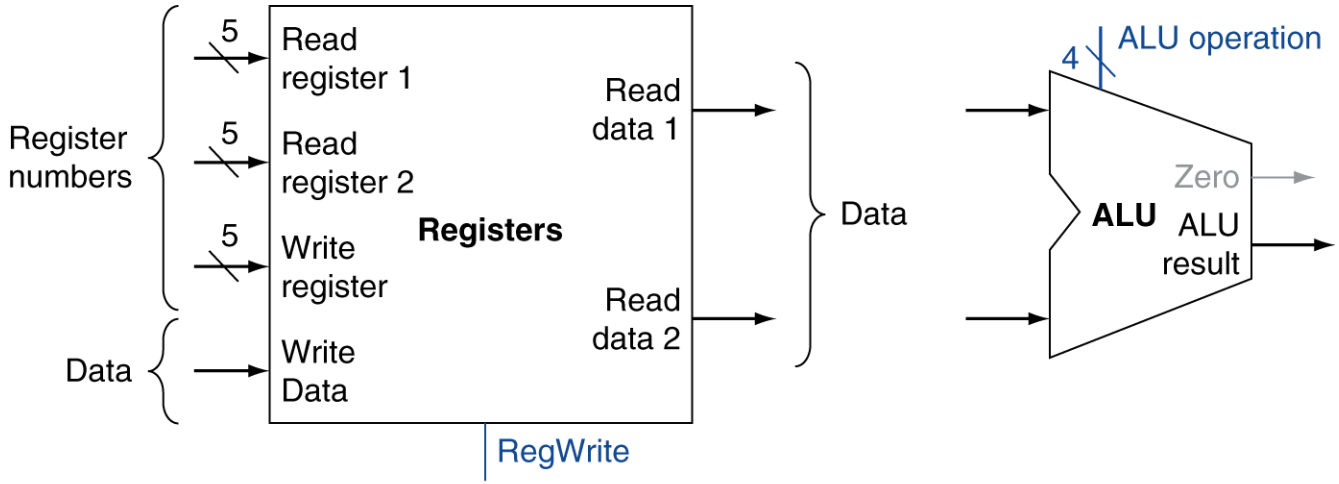
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

R-Format Instructions

R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

Examples: `add, sub, and, or, slt`
`"opcode" == 000000, "funct(5:0)"` decides which operation.

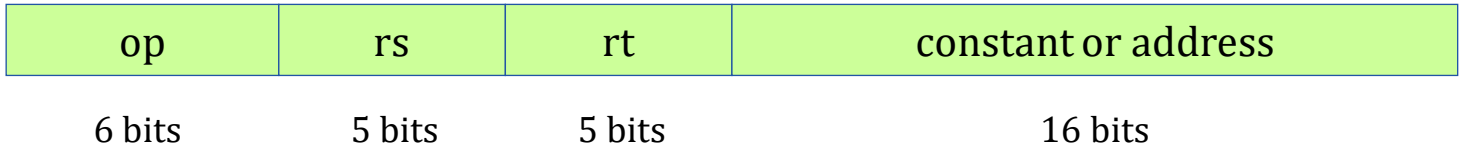


a. Registers

b. ALU

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: I-Format Instructions



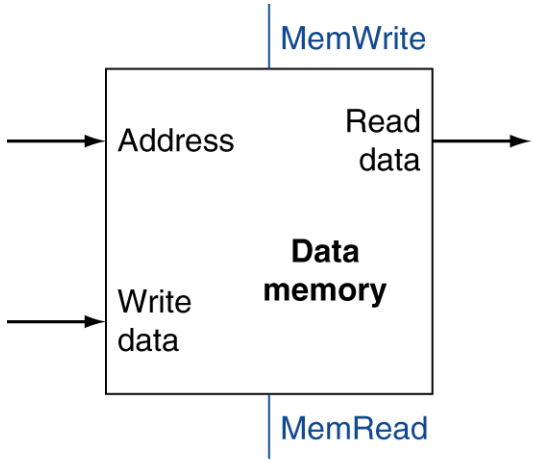
- Immediate arithmetic and load/store instructions
  - **rt** : destination or source register number
  - **constant** : -2^{15} to $+2^{15}-1$
  - **address** : offset added to base address in **rs** (e.g., lw, sw)
- Examples : `addi, lw, sw, beq, bne, slti, andi, ori, lui, …`

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

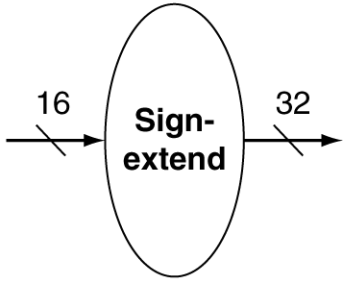
Load/Store Instructions

I-Format Instructions

- Read register operands
- Calculate address using 16-bit offset
  - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit



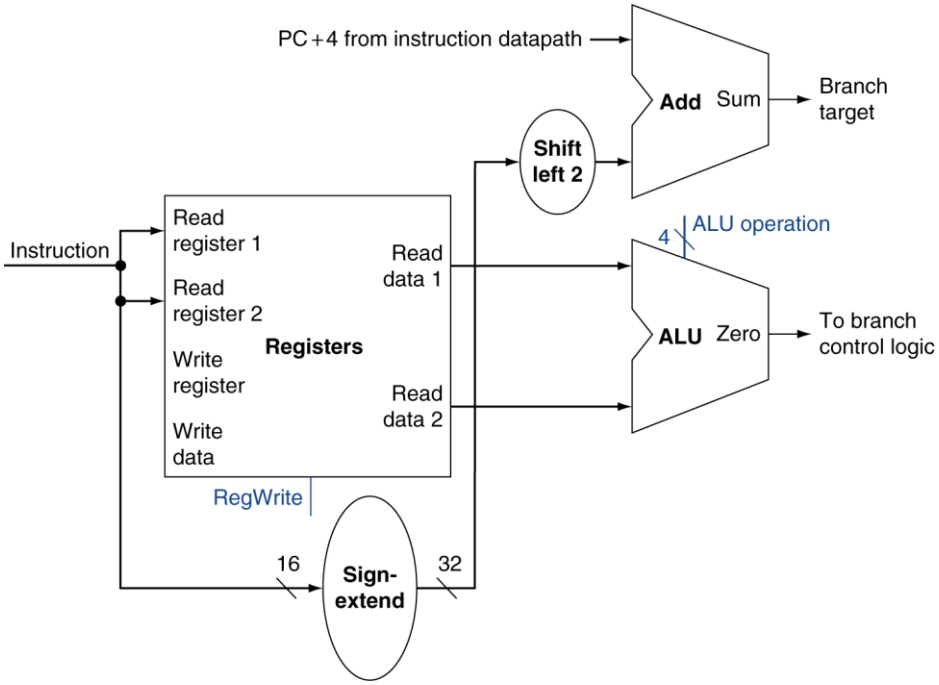
b. Sign extension unit

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Branch Instructions

I-Format Instructions

- Read register operands
- Calculate address using 16-bit offset
  - Use ALU, subtract and check Zero output
- Calculate Target Address
  - Sign-extend displacement
  - Shift-left 2 places (word displacement)
  - Add to PC+4 (Already calculated by instruction fetch)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

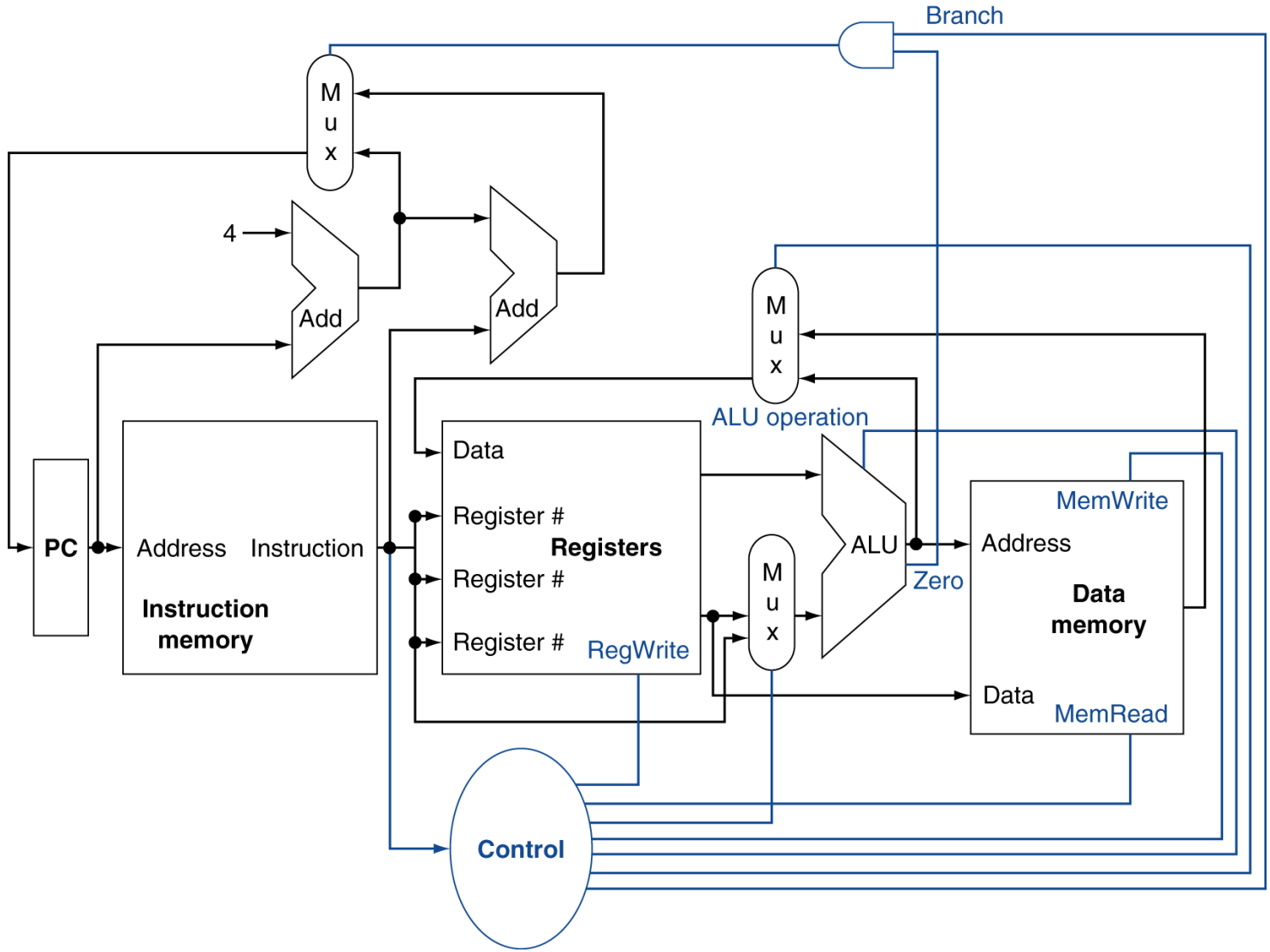
Composing the Elements

- **First-cut datapath does an instruction in one clock cycle**
  - Each datapath element can only do one function at a time
  - Hence, we need separate instruction and data memories

- **Use multiplexers where alternate data sources are used for different instructions**
  1. ALU input from… 'register' or 'immediate field of instruction'?
  2. Register write data from… 'ALU output' or 'data memory'?
  3. Next PC input from… 'PC+4 output' or 'branch target address'?
  4. Write register number from… 'Rt' or 'Rd'?

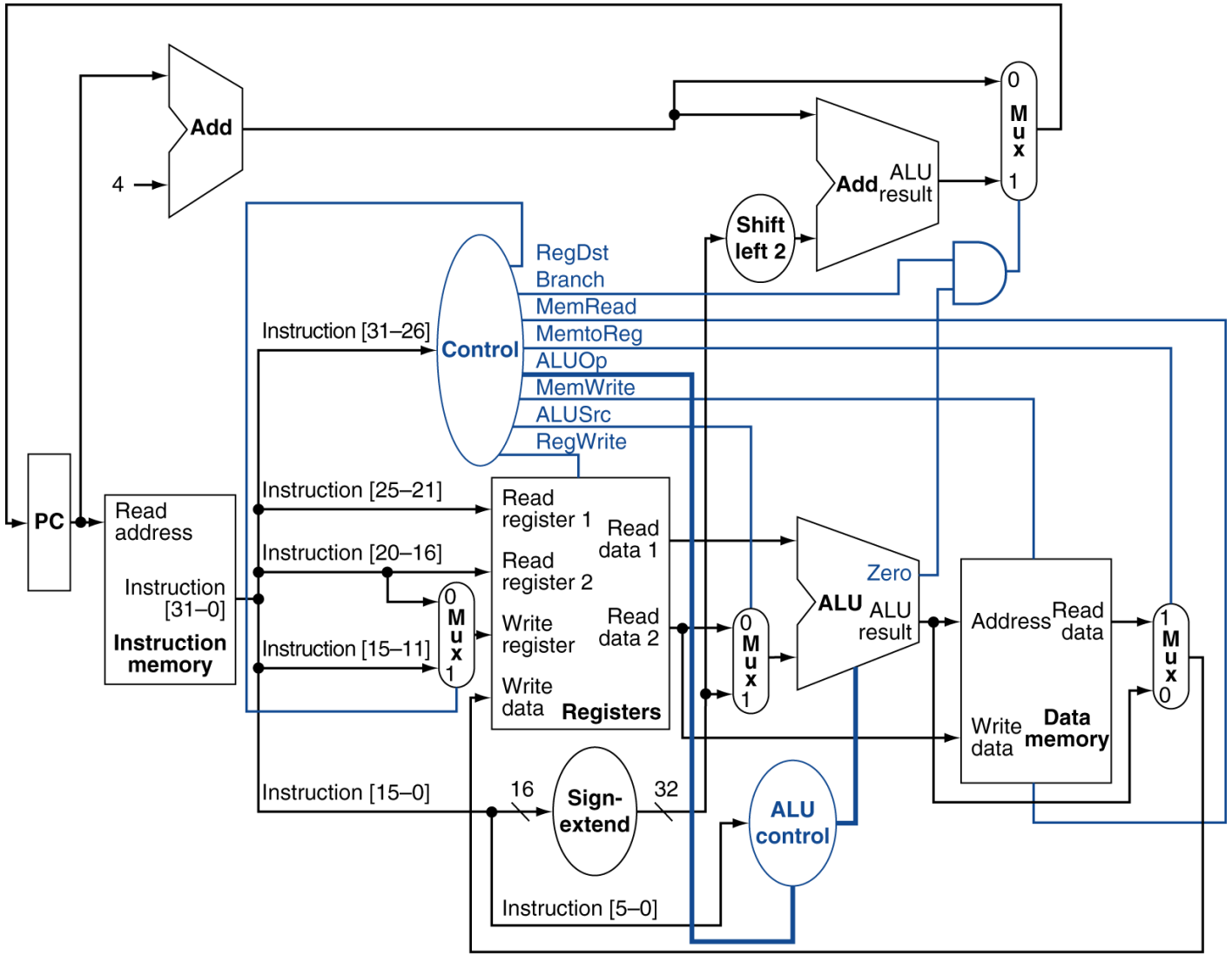
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

CPU Overview: Control



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

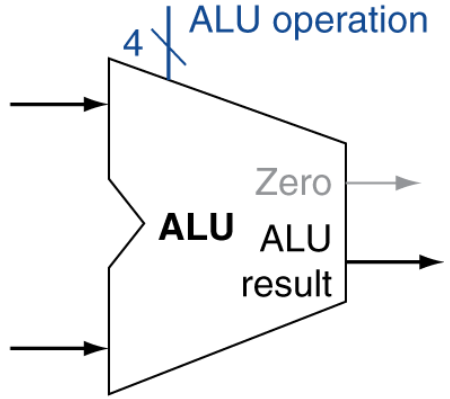
Full Datapath (with Control)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

ALU Control

- ALU used for
  - Load/Store: F = add
  - Branch: F = subtract
  - R-type: F depends on funct field



ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set-on-less-than
1100	NOR

ALU control

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

ALU Control

- Assume 2-bit ALUOp derived from opcode
  - Combinational logic derives ALU control

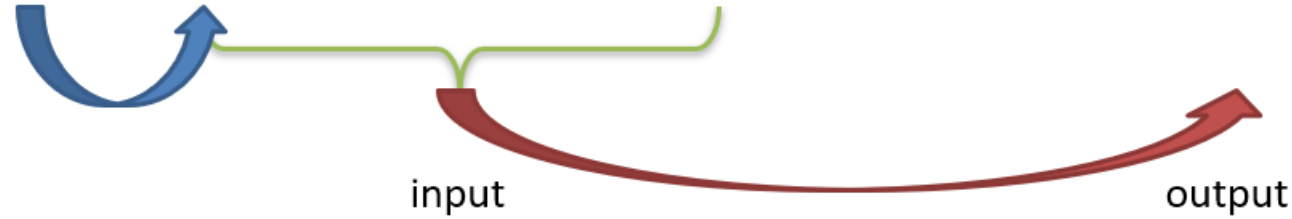
opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

ALU Control

- Assume 2-bit ALUOp derived from opcode
  - Combinational logic derives ALU control

instruction	opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	100011	00	load word	XXXXXX	add	0010
sw	101011	00	store word	XXXXXX	add	0010
beq	000100	01	branch equal	XXXXXX	subtract	0110
R-type	000000	10	add	100000	add	0010
			subtract	100010	subtract	0110
			AND	100100	AND	0000
			OR	100101	OR	0001
			set-on-less-than	101010	set-on-less-than	0111



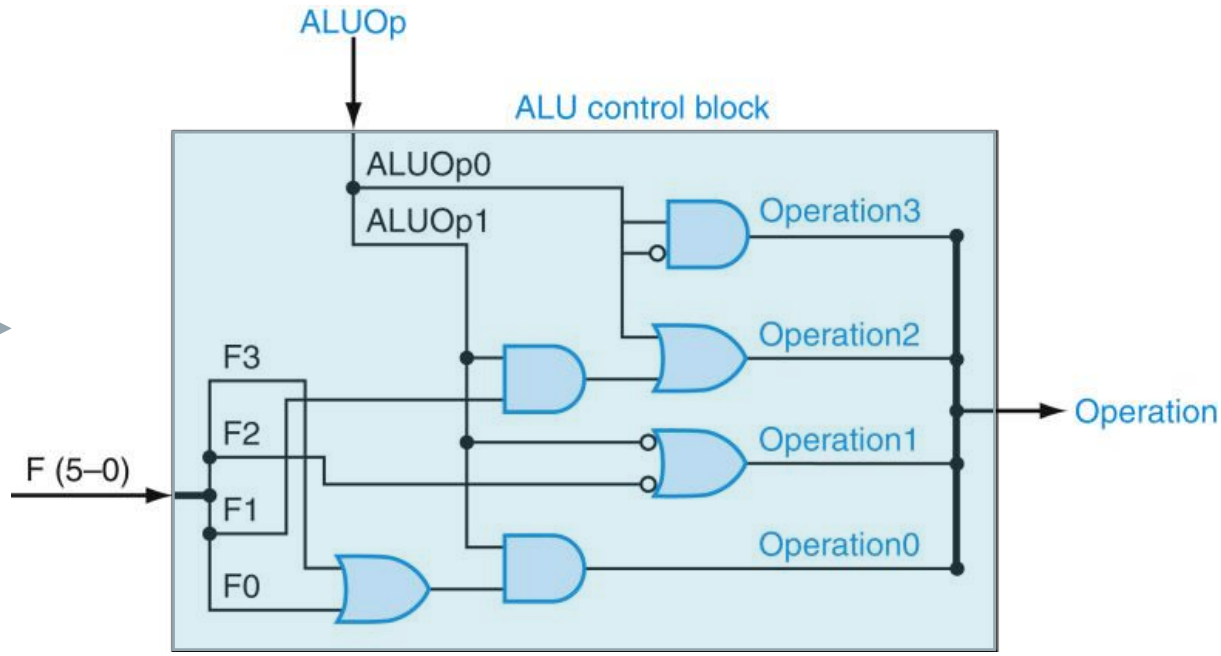
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

ALU Control

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
0	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

Karnough-Map!

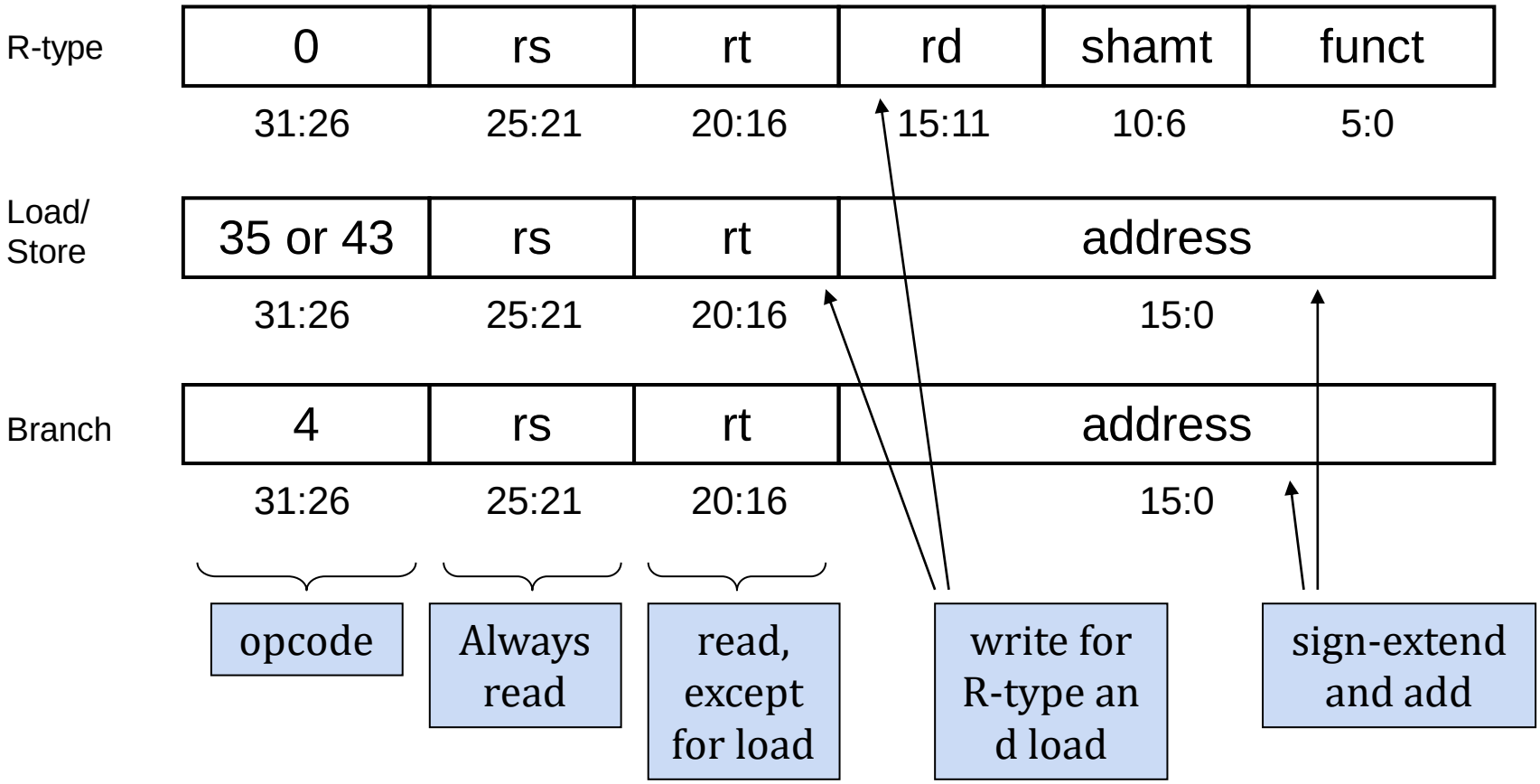
Op3 = always 0
= ALUOp0 · APUOp0
Op2 = ALUOp0
+ ALUOp1 · F1
Op1 = ALUOp1 + F2
Op0 = ALUOp1 · (F3 + F0)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

The Main Control Unit

- Control signals derived from instruction



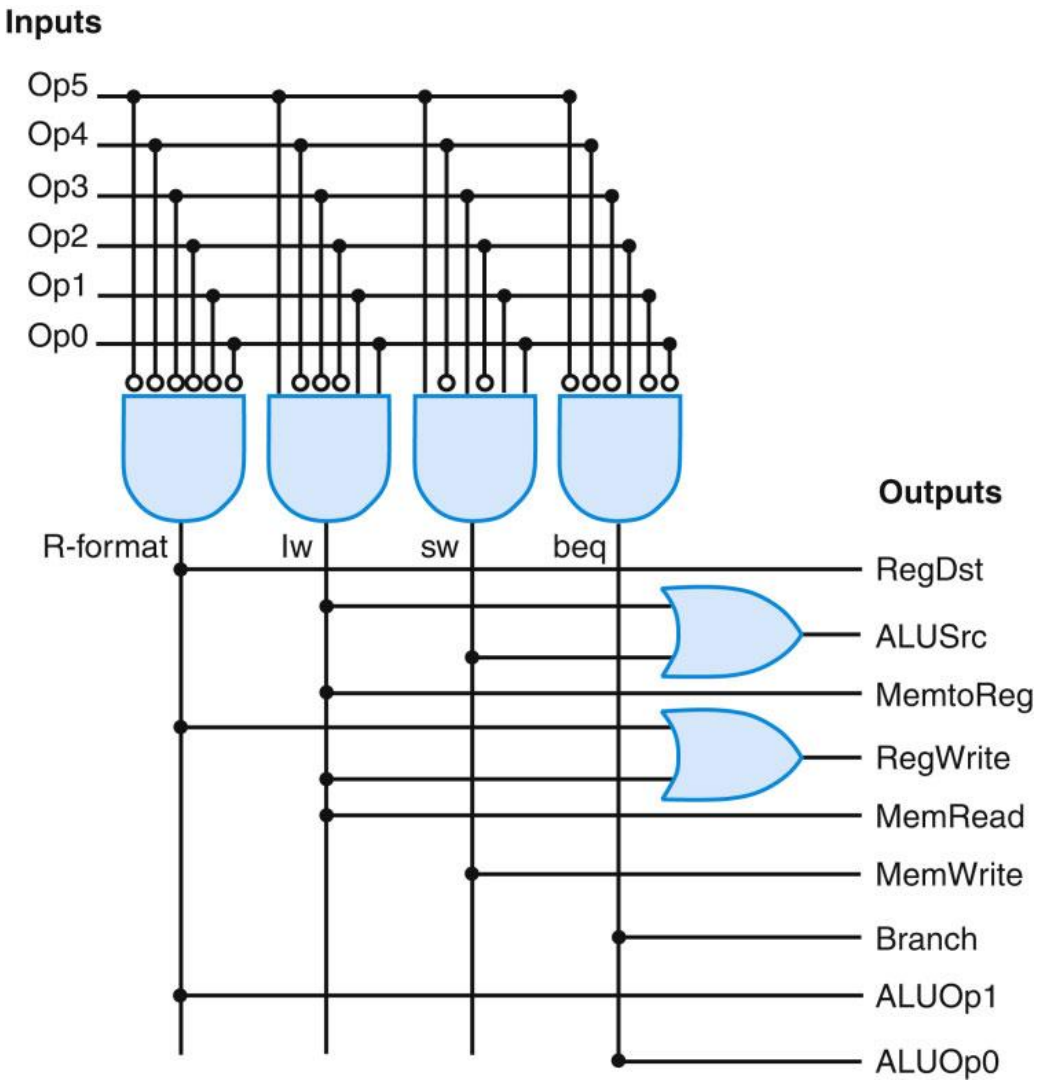
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Control

Control	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

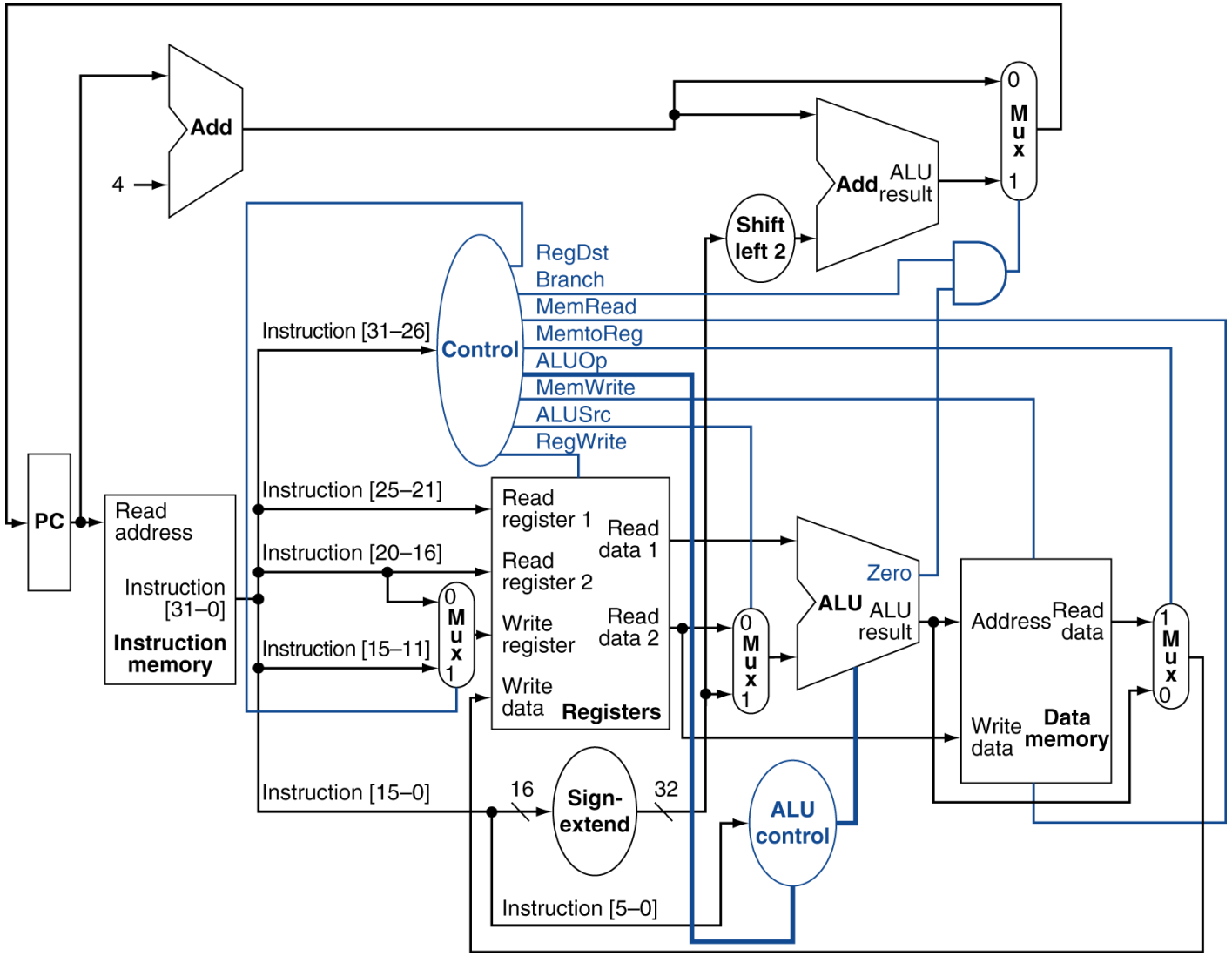
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Control



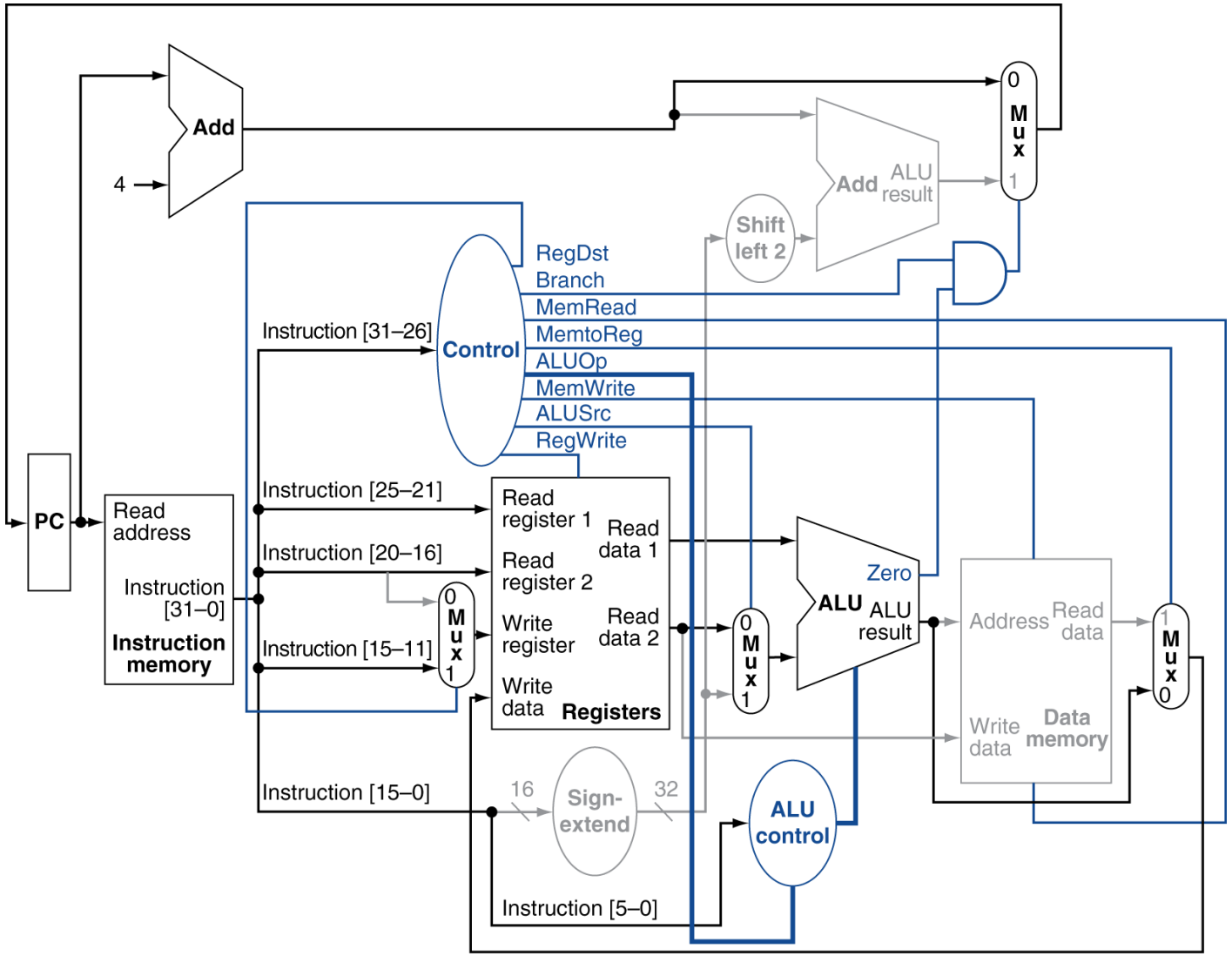
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Datapath with Control



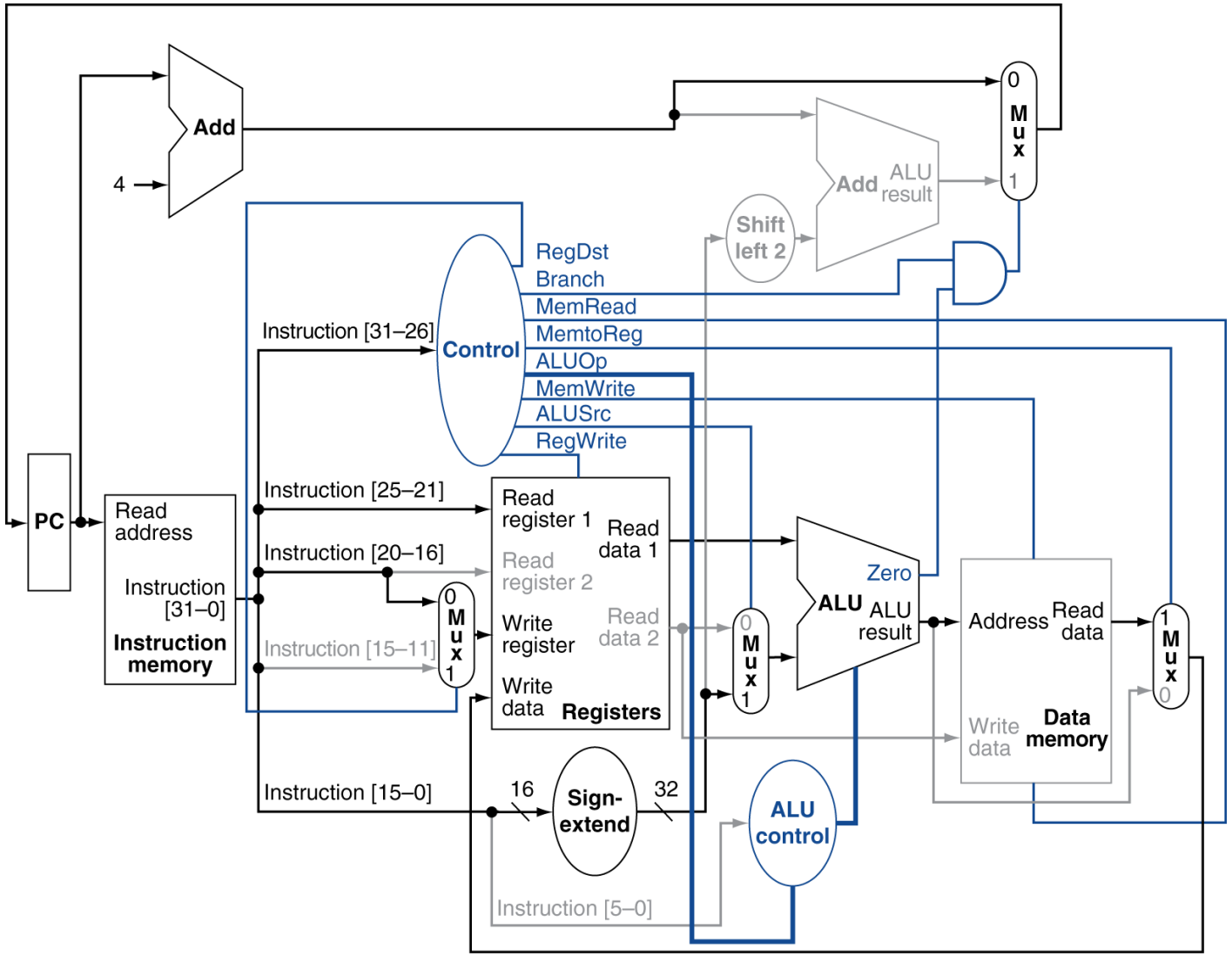
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Datapath with Control: R-Type Instruction



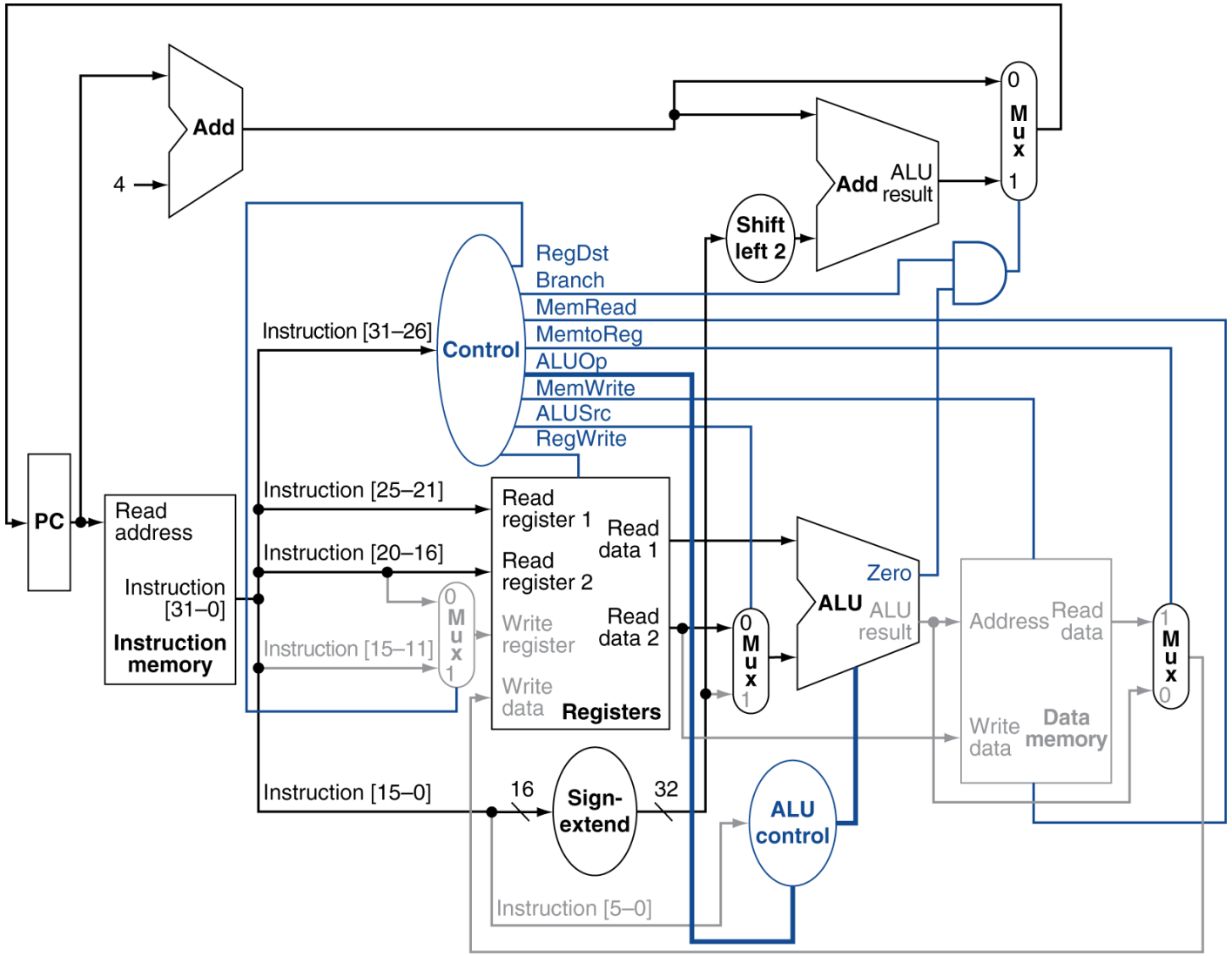
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Datapath with Control: Load Instruction



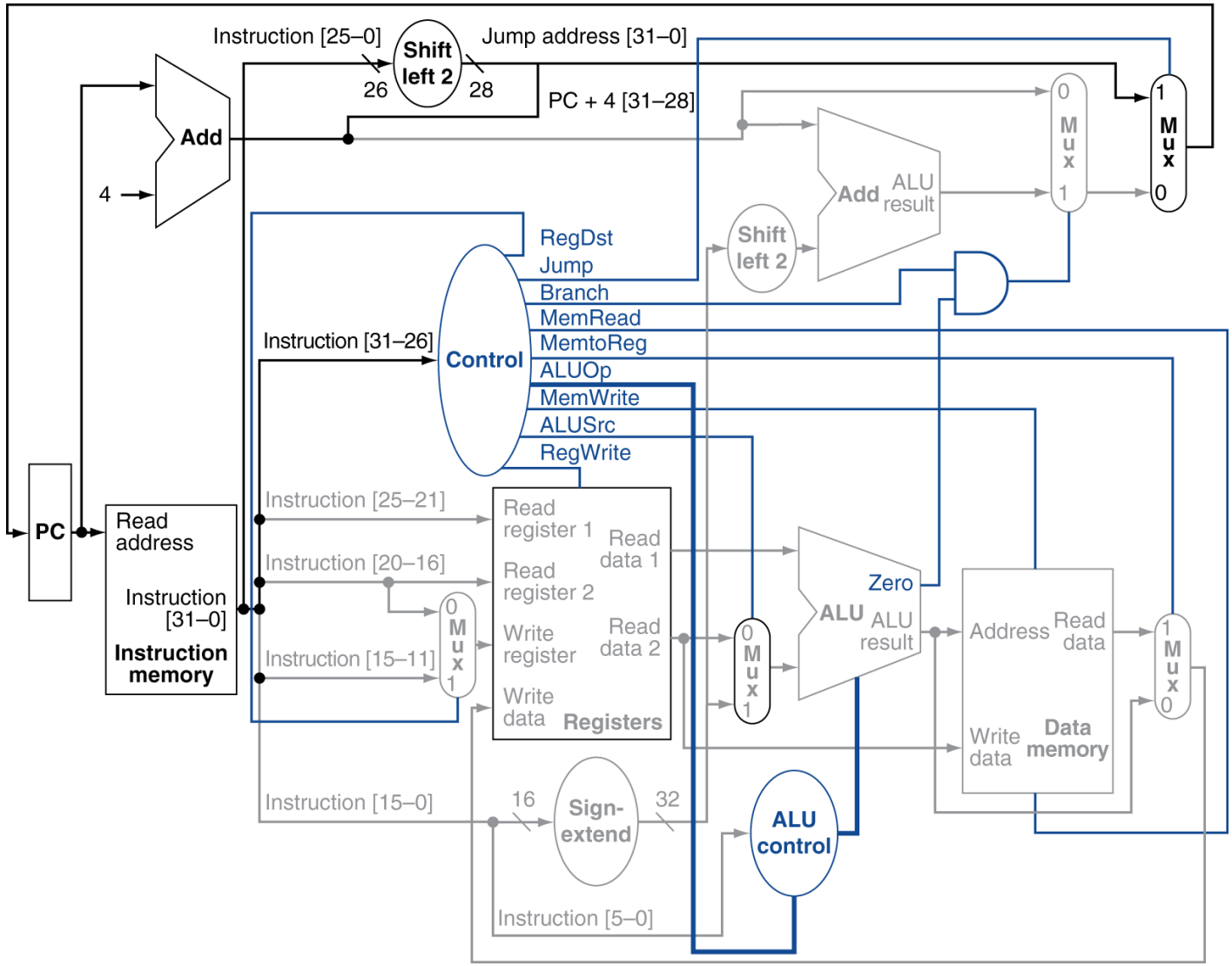
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Datapath with Control: Branch on Equal Instruction



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Datapath with Control: Jump Instruction



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential



Thank you

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430